

 Lab_4_VibeCoding_Student_handout.pdf

https://github.com/SETAPESU26/23_dots_and_boxes.git I have to go through this github repo, fork it and do the stuff they asked to do. Please layout the info needed to achieve this

* I read the handout and cloned the repo. I also played a few scripted games, so I could tell you what's actually broken before you start.

What the repo is

File	Role
main.py	Entry point: <code>DotsAndBoxes().run()</code>
game.py	Game loop, input parsing, scores, turns, winner
board.py	Line storage, finding completed boxes, end check, drawing the board
rules.py	<code>valid_move()</code> , <code>completed_boxes()</code>

What I found when I ran it

- **The board drawing is wrong.** Horizontal rows print as `---.---`, with no dots at the outer corners, while vertical rows print as `| | |`. The lines don't line up with the walls.
- **Completed boxes show X instead of the owner.**
- **Crash at end of input.** Ctrl+D / Ctrl+C raise a traceback.
- **Input checks are weak.** `-1` is rejected only as "must be numbers", there's no help or quit command.
- **The score and turn logic worked correctly in my tests.** Closing a box gave a point and another move; a line closing two boxes gave +2.

The four tasks (from the README)

1. Find and fix the bug, keeping the modular structure.
2. Add a real gameplay feature that touches more than one module.
3. Make input handling safe: malformed commands, bad coordinates, repeated lines, moves after the game is over.
4. Add automated tests (valid H move, valid V move, invalid/repeated move, box completion, end of game) and document the changes.

Commit once per task, and don't open a PR against SETAPESU26.

C:\Users\deli\23_dots_and_boxes>type board.py game.py rules.py

```
# board.py
class Board:
    def __init__(self, rows=2, cols=2):
        self.rows = rows
        self.cols = cols
        self.horizontal = [[False] * cols for _ in range(rows + 1)]
        self.vertical = [[False] * (cols + 1) for _ in range(rows)]
        self.completed = set()

    def add_line(self, orientation, row, col):
        if orientation == "H":
            self.horizontal[row][col] = True
        else:
            self.vertical[row][col] = True
        self._update_completed()

    def _update_completed(self):
        for r in range(self.rows):
            for c in range(self.cols):
                if (
                    self.horizontal[r][c]
                    and self.horizontal[r + 1][c]
                    and self.vertical[r][c]
                    and self.vertical[r][c + 1]
                ):
                    self.completed.add((r, c))

    def is_complete(self):
        total = self.rows * (self.cols + 1) + self.cols * (self.rows + 1)
        used = sum(map(sum, self.horizontal)) + sum(map(sum, self.vertical))
        return used == total

    def display(self, scores, current):
        print()
        print(f"Scores: P1={scores[0]} P2={scores[1]} | Turn: P{current + 1}")

        for r in range(self.rows + 1):
            print(".".join("---" if self.horizontal[r][c] else " " for c in range(self.cols)))
```

```

        if r < self.rows:
            middle = []
            for c in range(self.cols + 1):
                wall = "|" if self.vertical[r][c] else " "
                middle.append(wall)
                if c < self.cols:
                    middle.append(" " + ("X" if (r, c) in self.completed else " ") + " ")
            print("".join(middle))
        print()

# game.py
from board import Board
from rules import valid_move, completed_boxes

class DotsAndBoxes:
    def __init__(self):
        self.board = Board()
        self.current = 0
        self.scores = [0, 0]

    def run(self):
        print("Dots and Boxes")
        print("Enter moves as H row col or V row col.")
        print("Rows and columns start at 0.")
        print("Example: H 0 1")

        while not self.board.is_complete():
            self.board.display(self.scores, self.current)
            raw = input(f"Player {self.current + 1}, move: ").strip().upper()
            parts = raw.split()

            if len(parts) != 3:
                print("Invalid format.")
                continue

            orientation, row, col = parts
            if not row.isdigit() or not col.isdigit():
                print("Row and column must be numbers.")
                continue

            row, col = int(row), int(col)
            if not valid_move(self.board, orientation, row, col):
                print("Invalid or already-used move.")
                continue

            before = set(self.board.completed)
            self.board.add_line(orientation, row, col)
            newly_completed = completed_boxes(self.board, before)

            if newly_completed:
                self.scores[self.current] += newly_completed
                print(f"Player {self.current + 1} completed {newly_completed} box(es) and plays again.")
            else:
                self.current = 1 - self.current

            self.board.display(self.scores, self.current)
            print("Game over!")
            if self.scores[0] == self.scores[1]:
                print("The game is a draw.")
            else:
                winner = 1 if self.scores[0] > self.scores[1] else 2
                print(f"Player {winner} wins!")

# rules.py
def valid_move(board, orientation, row, col):
    if orientation not in {"H", "V"}:
        return False

    if orientation == "H":
        return (
            0 <= row <= board.rows
            and 0 <= col < board.cols
            and not board.horizontal[row][col]
        )

    return (
        0 <= row < board.rows
        and 0 <= col <= board.cols
        and not board.vertical[row][col]
    )

def completed_boxes(board, before):
    return len(board.completed) - before

```

✱ Scoring and turns already work correctly: closing a box gives +1 and the same player moves again. The real bug is in `Board.display()` in **board.py**. Each horizontal row was printed with dots only *between* the lines and none at the two ends, so every row was one character short and didn't line up with the walls.

Fix: in `board.py`, replace the whole `display` method with this:

```
def display(self, scores, current):
    print()
    print(f"Scores: P1={scores[0]} P2={scores[1]} | Turn: P{current + 1}")

    for r in range(self.rows + 1):
        # Draw dots at every corner (including both ends) so lines align with walls
        line = "."
        for c in range(self.cols):
            line += ("---" if self.horizontal[r][c] else " ") + "."
        print(line)
        if r < self.rows:
            middle = []
            for c in range(self.cols + 1):
                wall = "|" if self.vertical[r][c] else " "
                middle.append(wall)
                if c < self.cols:
                    middle.append(" " + ("X" if (r, c) in self.completed else " ") + " ")
            print("".join(middle))
    print()
```

Result after the same four moves:

```
.....
| X |
.....
. . .
```

Test and commit:

```
python main.py
git add board.py
git commit -m "Task 1: Fix misaligned board display (missing edge dots)"
```

```

.
Player 2, move:   H 9 9
Invalid or already-used move.

Scores: P1=0 P2=1 | Turn: P2
---.
| X |
---.
.

Player 2, move:
```

this is the output

* That's still the old output. The rows begin with `---` with no dot in front, so the new code isn't running yet. Most likely `board.py` wasn't saved.

The easiest fix is to replace the whole file: run `notepad board.py`, press `Ctrl+A`, delete, paste this, and save with `Ctrl+S`:

```
class Board:
    def __init__(self, rows=2, cols=2):
        self.rows = rows
        self.cols = cols
        self.horizontal = [[False] * cols for _ in range(rows + 1)]
        self.vertical = [[False] * (cols + 1) for _ in range(rows)]
        self.completed = set()

    def add_line(self, orientation, row, col):
        if orientation == "H":
            self.horizontal[row][col] = True
        else:
            self.vertical[row][col] = True
        self._update_completed()

    def _update_completed(self):
        for r in range(self.rows):
            for c in range(self.cols):
                if (
                    self.horizontal[r][c]
                    and self.horizontal[r + 1][c]
                    and self.vertical[r][c]
                    and self.vertical[r][c + 1]
                ):
                    self.completed.add((r, c))

    def is_complete(self):
```

```

total = self.rows * (self.cols + 1) + self.cols * (self.rows + 1)
used = sum(map(sum, self.horizontal)) + sum(map(sum, self.vertical))
return used == total

def display(self, scores, current):
    print()
    print(f"Scores: P1={scores[0]} P2={scores[1]} | Turn: P{current + 1}")

    for r in range(self.rows + 1):
        # Draw dots at every corner (including both ends) so lines align with walls
        line = "."
        for c in range(self.cols):
            line += ("---" if self.horizontal[r][c] else " ") + "."
        print(line)
        if r < self.rows:
            middle = []
            for c in range(self.cols + 1):
                wall = "|" if self.vertical[r][c] else " "
                middle.append(wall)
                if c < self.cols:
                    middle.append(" " + ("X" if (r, c) in self.completed else " ") + " ")
            print("".join(middle))
    print()

```

```
Scores: P1=0 P2=1 | Turn: P2
```

```

.---.
| X |
.---.

```

```

Player 2, move: H 9 9
Invalid or already-used move.

```

```
Scores: P1=0 P2=1 | Turn: P2
```

```

.---.
| X |
.---.

```

now it worked

* Task 1 is fixed. Commit it first.

Task 2: Board size choice and box owners

This feature changes all three modules:

- **rules.py** reads and checks the board size the player types.
- **board.py** records which player claimed each box, shows 1 or 2 instead of X, and prints row and column numbers.
- **game.py** asks for the board size at the start and tells the board who made each move.

board.py

```

class Board:
    def __init__(self, rows=2, cols=2):
        self.rows = rows
        self.cols = cols
        self.horizontal = [[False] * cols for _ in range(rows + 1)]
        self.vertical = [[False] * (cols + 1) for _ in range(rows)]
        # Maps each completed box (row, col) to the player index (0 or 1) who claimed it
        self.completed = {}

    def add_line(self, orientation, row, col, player=0):
        if orientation == "H":
            self.horizontal[row][col] = True
        else:
            self.vertical[row][col] = True
        self._update_completed(player)

    def _update_completed(self, player):
        for r in range(self.rows):
            for c in range(self.cols):
                if (
                    (r, c) not in self.completed
                    and self.horizontal[r][c]
                    and self.horizontal[r + 1][c]

```

```

        and self.vertical[r][c]
        and self.vertical[r][c + 1]
    ):
        self.completed[(r, c)] = player

def is_complete(self):
    total = self.rows * (self.cols + 1) + self.cols * (self.rows + 1)
    used = sum(map(sum, self.horizontal)) + sum(map(sum, self.vertical))
    return used == total

def display(self, scores, current):
    print()
    print(f"Scores: P1={scores[0]} P2={scores[1]} | Turn: P{current + 1}")

    # Column numbers for horizontal lines
    print(" " + "".join(f" {c} " for c in range(self.cols)))
    for r in range(self.rows + 1):
        # Draw a dot at every corner (including both ends) so lines align with walls
        line = f"{r:>2} ."
        for c in range(self.cols):
            line += ("---" if self.horizontal[r][c] else " ") + "."
        print(line)
        if r < self.rows:
            middle = [" "]
            for c in range(self.cols + 1):
                wall = "|" if self.vertical[r][c] else " "
                middle.append(wall)
                if c < self.cols:
                    owner = self.completed.get((r, c))
                    mark = " " if owner is None else str(owner + 1)
                    middle.append(f" {mark} ")
            print("".join(middle))
    print()

```

rules.py

```

MIN_SIZE = 1
MAX_SIZE = 5

def valid_move(board, orientation, row, col):
    if orientation not in {"H", "V"}:
        return False

    if orientation == "H":
        return (
            0 <= row <= board.rows
            and 0 <= col < board.cols
            and not board.horizontal[row][col]
        )

    return (
        0 <= row < board.rows
        and 0 <= col <= board.cols
        and not board.vertical[row][col]
    )

def completed_boxes(board, before):
    return len(set(board.completed) - set(before))

def parse_board_size(text):
    """Parse 'rows cols' (e.g. '3 3'). Returns (rows, cols) or None if invalid.
    Empty input means the default 2x2 board."""
    text = text.strip().lower().replace("x", " ")
    if not text:
        return 2, 2
    parts = text.split()
    if len(parts) != 2 or not all(p.isdigit() for p in parts):
        return None
    rows, cols = int(parts[0]), int(parts[1])
    if not (MIN_SIZE <= rows <= MAX_SIZE and MIN_SIZE <= cols <= MAX_SIZE):
        return None
    return rows, cols

```

game.py

```

from board import Board
from rules import valid_move, completed_boxes, parse_board_size, MIN_SIZE, MAX_SIZE

class DotsAndBoxes:
    def __init__(self, rows=2, cols=2):
        self.board = Board(rows, cols)
        self.current = 0
        self.scores = [0, 0]

    def choose_board_size(self):
        while True:
            raw = input(f"Board size as 'rows cols' ({MIN_SIZE}-{MAX_SIZE}, Enter for 2 2): ")
            size = parse_board_size(raw)
            if size:
                self.board = Board(*size)

```

```

        return
        print(f"Please enter two numbers between {MIN_SIZE} and {MAX_SIZE}, e.g. 3 3.")

def run(self):
    print("Dots and Boxes")
    print("Enter moves as H row col or V row col.")
    print("Rows and columns start at 0.")
    print("Example: H 0 1")
    self.choose_board_size()

    while not self.board.is_complete():
        self.board.display(self.scores, self.current)
        raw = input(f"Player {self.current + 1}, move: ").strip().upper()
        parts = raw.split()

        if len(parts) != 3:
            print("Invalid format.")
            continue

        orientation, row, col = parts
        if not row.isdigit() or not col.isdigit():
            print("Row and column must be numbers.")
            continue

        row, col = int(row), int(col)
        if not valid_move(self.board, orientation, row, col):
            print("Invalid or already-used move.")
            continue

        before = set(self.board.completed)
        self.board.add_line(orientation, row, col, self.current)
        newly_completed = completed_boxes(self.board, before)

        if newly_completed:
            self.scores[self.current] += newly_completed
            print(f"Player {self.current + 1} completed {newly_completed} box(es) and plays again.")
        else:
            self.current = 1 - self.current

    self.board.display(self.scores, self.current)
    print("Game over!")
    if self.scores[0] == self.scores[1]:
        print("The game is a draw.")
    else:
        winner = 1 if self.scores[0] > self.scores[1] else 2
        print(f"Player {winner} wins!")

```

Test it: run `python main.py`, type `3 3`, then play `H 0 0`, `V 0 0`, `H 1 0`, `V 0 1`. The box should show **2**.

```

git add .
git commit -m "Task 2: Board size selection and box ownership display"

```

```

Player 2, move:   H 9 9
Invalid or already-used move.

Scores: P1=0  P2=1 | Turn: P2
      0   1   2
0  .---.   .   .
   | 2 |   .   .
1  .---.   .   .

2  .   .   .   .
3  .   .   .   .

Player 2, move:

```

final output

✳ Task 2 works. Commit it.

Task 3: Safe input handling

- **rules.py** gets `parse_move()`, which returns a clear error for bad format, wrong direction, non-numbers or negatives, out-of-range lines, lines already drawn, and moves after the game is over.
- **board.py**'s `add_line()` refuses illegal moves itself, so the board can't be corrupted.
- **game.py** adds `help` and `quit`, accepts lowercase moves, and ends cleanly on Ctrl+C or Ctrl+Z.

rules.py – add at the bottom:

```

def parse_move(board, text):
    """Turn raw input like 'h 0 1' into (orientation, row, col).

```

```

Returns (move, None) on success or (None, error_message) on failure.
Never changes the board."""
if board.is_complete():
    return None, "The board is already complete. No more moves can be made."

parts = text.strip().upper().split()
if len(parts) != 3:
    return None, "Invalid format. Use: H row col or V row col (e.g. H 0 1).\"

orientation, row_text, col_text = parts
if orientation not in {"H", "V"}:
    return None, "Direction must be H (horizontal) or V (vertical).\"

try:
    row, col = int(row_text), int(col_text)
except ValueError:
    return None, "Row and column must be whole numbers.\"

if orientation == \"H\":
    max_row, max_col = board.rows, board.cols - 1
else:
    max_row, max_col = board.rows - 1, board.cols
if not (0 <= row <= max_row and 0 <= col <= max_col):
    return None, (f\"Out of range. For {orientation} lines, row must be 0-{max_row} \"
                f\"and column 0-{max_col}.\")

if not valid_move(board, orientation, row, col):
    return None, \"That line has already been drawn. Choose another.\"

return (orientation, row, col), None

```

board.py – replace `add_line` :

```

def add_line(self, orientation, row, col, player=0):
    # Defensive check: never let a bad call corrupt the board
    if orientation == \"H\":
        if not (0 <= row <= self.rows and 0 <= col < self.cols) or self.horizontal[row][col]:
            raise ValueError(f\"Illegal move: {orientation} {row} {col}\")
    elif orientation == \"V\":
        if not (0 <= row < self.rows and 0 <= col <= self.cols) or self.vertical[row][col]:
            raise ValueError(f\"Illegal move: {orientation} {row} {col}\")
    else:
        raise ValueError(f\"Illegal orientation: {orientation}\")

    if orientation == \"H\":
        self.horizontal[row][col] = True
    else:
        self.vertical[row][col] = True
    self._update_completed(player)

```

game.py – replace the whole file:

```

from board import Board
from rules import completed_boxes, parse_board_size, parse_move, MIN_SIZE, MAX_SIZE

HELP_TEXT = """Commands:
H row col   draw a horizontal line (e.g. H 0 1)
V row col   draw a vertical line   (e.g. V 1 0)
help        show this help
quit        end the game"""

class DotsAndBoxes:
    def __init__(self, rows=2, cols=2):
        self.board = Board(rows, cols)
        self.current = 0
        self.scores = [0, 0]

    def choose_board_size(self):
        while True:
            raw = input(f\"Board size as 'rows cols' ({MIN_SIZE}-{MAX_SIZE}, Enter for 2 2): \")
            size = parse_board_size(raw)
            if size:
                self.board = Board(*size)
                return
            print(f\"Please enter two numbers between {MIN_SIZE} and {MAX_SIZE}, e.g. 3 3.\")

    def play_move(self, orientation, row, col):
        """Apply a validated move and update score/turn. Returns boxes completed."""
        before = set(self.board.completed)
        self.board.add_line(orientation, row, col, self.current)
        newly_completed = completed_boxes(self.board, before)

        if newly_completed:
            self.scores[self.current] += newly_completed
        else:
            self.current = 1 - self.current
        return newly_completed

    def run(self):
        print(\"Dots and Boxes\")
        print(\"Enter moves as H row col or V row col.\")
        print(\"Rows and columns start at 0.\")

```

```

print("Example: H 0 1 (type 'help' for commands, 'quit' to exit)")
try:
    self.choose_board_size()
    self.game_loop()
except (EOFError, KeyboardInterrupt):
    print("\nGame ended by user.")

def game_loop(self):
    while not self.board.is_complete():
        self.board.display(self.scores, self.current)
        raw = input(f"Player {self.current + 1}, move: ").strip()

        if raw.lower() in {"q", "quit", "exit"}:
            print("Game ended by user.")
            return
        if raw.lower() in {"h", "help", "?"}:
            print(HELP_TEXT)
            continue

        move, error = parse_move(self.board, raw)
        if error:
            print(error)
            continue

        player = self.current
        newly_completed = self.play_move(*move)
        if newly_completed:
            print(f"Player {player + 1} completed {newly_completed} box(es) and plays again.")

    self.board.display(self.scores, self.current)
    print("Game over!")
    if self.scores[0] == self.scores[1]:
        print("The game is a draw.")
    else:
        winner = 1 if self.scores[0] > self.scores[1] else 2
        print(f"Player {winner} wins!")

```

```

git add .
git commit -m "Task 3: Input validation and robust game-state handling"

```

what to type after python main.py

* The game first asks for the board size. Press **Enter** for 2x2, then at `Player 1, move:` paste:

```

hello
X 0 0
H a b
H -1 0
h 0 0
H 0 0
help

```

You typed	Game says
hello	Invalid format
X 0 0	Direction must be H or V
H a b	Row and column must be whole numbers
H -1 0	Out of range
h 0 0	accepted, turn passes to Player 2
H 0 0	That line has already been drawn
help	Shows the list of commands

Type `quit` to exit.

```

Player 2, move: help
Commands:
  H row col  draw a horizontal line (e.g. H 0 1)
  V row col  draw a vertical line   (e.g. V 1 0)
  help       show this help
  quit       end the game

Scores: P1=0  P2=0 | Turn: P2
  0  1
0 .--. .
1 . . .
2 . . .

Player 2, move: |

```

designated output

* Task 3 works. Commit it.

Task 4: Tests and documentation

Save **test_game.py** next to `main.py`. It has 13 tests: valid horizontal and vertical moves, invalid and repeated moves, completing a box (and one line closing two boxes), the end of the game, and a check that bad input never changes the board.

 test_game.py

```

import unittest

from board import Board
from game import DotsAndBoxes
from rules import parse_move, parse_board_size


def play(game, *moves):
    """Play a list of moves like 'H 0 0' through the real validation + game logic."""
    for text in moves:
        move, error = parse_move(game.board, text)
        if error:
            raise AssertionError(f"{text!r} rejected: {error}")
        game.play_move(*move)


class TestMoves(unittest.TestCase):
    def setUp(self):
        self.game = DotsAndBoxes(2, 2)

    def test_valid_horizontal_move(self):
        play(self.game, "H 0 1")
        self.assertTrue(self.game.board.horizontal[0][1])
        self.assertEqual(self.game.current, 1) # no box -> turn passes
        self.assertEqual(self.game.scores, [0, 0])

    def test_valid_vertical_move(self):
        play(self.game, "V 1 2")
        self.assertTrue(self.game.board.vertical[1][2])
        self.assertEqual(self.game.current, 1)

    def test_lowercase_move_accepted(self):
        play(self.game, "v 0 0")
        self.assertTrue(self.game.board.vertical[0][0])

    def test_repeated_move_rejected(self):
        play(self.game, "H 0 0")
        move, error = parse_move(self.game.board, "H 0 0")
        self.assertIsNone(move)
        self.assertIn("already", error)

    def test_invalid_inputs_rejected(self):
        for text in ["", "hello", "X 0 0", "H a b", "H -1 0", "H 3 0", "V 0 3", "H 0 0 0"]:
            move, error = parse_move(self.game.board, text)
            self.assertIsNone(move, text)
            self.assertTrue(error, text)

    def test_invalid_input_does_not_change_state(self):
        play(self.game, "H 0 0")
        snapshot = ([r[:] for r in self.game.board.horizontal],
                    [r[:] for r in self.game.board.vertical],
                    self.game.scores[:], self.game.current)
        for text in ["H 0 0", "H 9 9", "junk"]:
            parse_move(self.game.board, text)
        self.assertEqual(snapshot, ([r[:] for r in self.game.board.horizontal],
                                    [r[:] for r in self.game.board.vertical],
                                    self.game.scores, self.game.current))

    def test_board_rejects_illegal_add_line(self):
        board = Board(2, 2)
        board.add_line("H", 0, 0)
        with self.assertRaises(ValueError):

```

```

        board.add_line("H", 0, 0)
    with self.assertRaises(ValueError):
        board.add_line("V", 5, 5)

class TestBoxes(unittest.TestCase):
    def test_completing_a_box_scores_and_keeps_turn(self):
        game = DotsAndBoxes(2, 2)
        play(game, "H 0 0", "V 0 0", "H 1 0") # P1, P2, P1
        self.assertEqual(game.current, 1)      # P2 to move
        play(game, "V 0 1")                   # P2 closes box (0,0)
        self.assertEqual(game.scores, [0, 1])
        self.assertEqual(game.current, 1)      # P2 plays again
        self.assertEqual(game.board.completed, {(0, 0): 1})

    def test_one_line_completing_two_boxes(self):
        game = DotsAndBoxes(2, 1)
        play(game, "H 0 0", "V 0 0", "V 0 1", "H 2 0", "V 1 0", "V 1 1")
        player = game.current
        play(game, "H 1 0") # middle line closes both boxes
        self.assertEqual(game.scores[player], 2)
        self.assertEqual(game.current, player)

class TestEndOfGame(unittest.TestCase):
    def test_game_ends_when_all_lines_drawn(self):
        game = DotsAndBoxes(1, 1)
        play(game, "H 0 0", "H 1 0", "V 0 0")
        self.assertFalse(game.board.is_complete())
        play(game, "V 0 1")
        self.assertTrue(game.board.is_complete())
        self.assertEqual(sum(game.scores), 1)

    def test_no_moves_after_board_complete(self):
        game = DotsAndBoxes(1, 1)
        play(game, "H 0 0", "H 1 0", "V 0 0", "V 0 1")
        move, error = parse_move(game.board, "H 0 0")
        self.assertIsNone(move)
        self.assertIn("complete", error)

    def test_full_2x2_game_scores_add_up(self):
        game = DotsAndBoxes(2, 2)
        play(game, "H 0 0", "H 0 1", "H 1 0", "H 1 1", "H 2 0", "H 2 1",
            "V 0 0", "V 0 1", "V 0 2", "V 1 0", "V 1 1", "V 1 2")
        self.assertTrue(game.board.is_complete())
        self.assertEqual(sum(game.scores), 4)

class TestBoardSize(unittest.TestCase):
    def test_parse_board_size(self):
        self.assertEqual(parse_board_size(""), (2, 2))
        self.assertEqual(parse_board_size("3 4"), (3, 4))
        self.assertEqual(parse_board_size("3x3"), (3, 3))
        for bad in ["0 2", "9 9", "a b", "3"]:
            self.assertIsNone(parse_board_size(bad), bad)

if __name__ == "__main__":
    unittest.main()

```

Run them with `python -m unittest -v`. You should see `Ran 13 tests ... OK`.

Then add a **Changes Made** section to the bottom of README.md describing each task and the design decisions: validation stays in `rules.py`, state in `board.py`, flow in `game.py`, `main.py` unchanged, score/turn logic moved into `play_move()` so it can be tested, and no third-party dependencies.

```

git add .
git commit -m "Task 4: Add unit tests and document changes"

```

give me a short one like 3 3 H 0

*

```

3 3
H 0 0
V 0 0
H 1 0
V 0 1
hello
H 9 9

```

Run `python main.py`, paste this at the board size prompt and press Enter. It shows the 3x3 board, the box owned by Player 2, and both error messages. Then type `quit`.